

DealFlow360 — Team Execution Plan

4-person team | Design Thinking + Agile | Mentor-friendly milestone breakdown

Team split assumed:

Team A (2 people): Backend + Frontend "scaffolding" — auth, CRUD, core screens, DB, API layer, infra

Team B (2 people): "Smart layer" — discount risk scoring, upsell engine, deal health/anomaly detection

PHASE 0 — Empathize & Define (Requirement Gathering)

Goal: Everyone agrees on exact scope before writing code. This is the phase mentors check first — show them this as a one-pager.

Checklist:

- ☐ Re-read the problem statement as a team, line by line, and write down every **explicit** requirement (ignore "optional/bonus" lines for now — A6 Upsell Setup and multi-currency are explicitly marked optional)
- ☐ List the 5 user roles and one sentence each on what they need to see and *do*
- ☐ Define your **MVP cut line**: which of the 9 modules (A1–A7, B1–B9) are "must demo" vs "nice to have"
- ☐ Write the 8-step Quick Test Flow (given in the doc) as your **Definition of Done** for the whole hackathon — if all 8 steps work, you're done
- ☐ Agree on what "connectable with existing modules" means for your build (see Phase 2 — this drives your architecture choice, not just a buzzword)

Deliverable to show mentor: One page with roles, MVP scope, and the 8-step test flow marked as your acceptance criteria.

PHASE 1 — Ideate & Prioritize (MoSCoW)

Break every module into Must / Should / Could so both sub-teams know what "done" looks like each sprint.

Priority	Modules
Must have	Login/Signup, Product & Price setup, Quotation Builder, Discount Tier + Approval routing, Approval screen, Warehouse split, Subscription/Billing screen, Customer Portal negotiation, Basic Deal Health dashboard
Should have	Upsell/cross-sell panel, Reporting filters & export, Audit trail UI
Could have (post-MVP)	Multi-currency, promotion tagging, advanced anomaly ML, notifications/email

Checklist:

- ☐ Convert each "Must have" into a user story: "As a [role], I can [action], so that [outcome]"
- ☐ Estimate story size (S/M/L) as a team
- ☐ Map stories to the 5 sprints below

PHASE 2 — Architecture Decisions (HLD) + Tech Stack Reasoning

The core architectural decision (this is what answers your mentor's "connectable + extendable" ask)

Build a **modular monolith with an internal event bus**, not a tangle of direct function calls, and not full microservices (too slow to set up in a hackathon).

Each module (Quotation, Approval, Fulfillment, Billing, Subscription) is a **self-contained service** with its own clear interface — even though they all run in one deployable app.

Modules **never call each other's internal code directly**. When something important happens (a quote is updated, an order is approved), the module **emits an event** (e.g. `QuotationUpdated`, `ApprovalRequired`, `OrderApproved`) onto an internal event bus.

The **Smart Layer (risk scoring, upsell, deal health)** is built entirely as **event consumers** — they listen to events and react, they are never called directly by the core layer.

Why this solves your mentor's requirement:

Want to add a new smart feature later (e.g. churn prediction)? Just add a new consumer that listens to existing events. Zero changes to Quotation/Approval/Billing code.

Want to swap Postgres for another DB, or split a module into its own microservice later? Because modules only talk via events/interfaces (not direct imports), you can extract any module without rewriting the others.

This is literally how real systems (Odoo's own bus, Salesforce Platform Events, Shopify webhooks) scale — you're mimicking production patterns, which is a strong point to make to mentors.

Recommended Tech Stack (and why)

Layer	Choice	Why
Frontend	React + TypeScript (Vite)	Component reuse across 3 UIs (Sales workspace, Customer portal, Admin) with shared design system; TS catches bugs fast under time pressure
API layer	Node.js + Express (or FastAPI if team prefers Python)	Fast to scaffold REST endpoints; same language as event bus logic if Node, reducing context switching for a 4-person team
Core business logic	Hand-written in application layer	The brief explicitly wants business logic built from scratch, not faked

Layer	Choice	Why
	(services/controllers) — no low-code/no-code, no third-party CPQ API	
Database	PostgreSQL	Your data is deeply relational (orders → lines → approvals → invoices, with strict integrity needs like "an approved quote must have an audit trail"). Postgres also supports JSONB columns, useful for flexible fields (e.g. variant attributes) without needing a second DB
Event bus	In-process EventEmitter (Node) or lightweight Redis Pub/Sub	Enough to prove the <i>pattern</i> in a hackathon. Redis pub/sub is a small addition that also doubles as your cache — and if you later need durability/replay, swapping to RabbitMQ/Kafka is a drop-in change because your modules only know about "publish event" / "subscribe to event", not the transport
Cache / session	Redis	Session tokens, and caching frequently-read data like price lists and stock levels for the fulfillment split calculation
Auth	JWT-based custom auth (not a third-party auth API, since brief wants internal logic)	Simple to build, demonstrates role-based access control (Rep/Manager/Finance/Customer/Admin)
Realtime updates (optional/should-have)	WebSockets (Socket.io)	For live margin updates on quote builder and live approval status without polling

Why NOT microservices for the hackathon: setup/deployment overhead (service discovery, inter-service auth, multiple repos) eats your limited time without adding visible value in a 2–3 day demo. The event-bus-in-a-monolith pattern gives you 90% of the "future scalability" story mentors want to hear, at 10% of the setup cost.

Checklist:

- ☐ Finalize stack (lock this before Sprint 1 — don't debate mid-build)
- ☐ Set up repo structure: `/frontend`, `/backend/modules/{quotation,approval,fulfillment,billing,subscription}`, `/backend/smart-layer/{risk-engine,upsell-engine,deal-health}`, `/backend/event-bus`
- ☐ Agree on event naming convention (e.g. `PascalCase` past-tense: `QuotationCreated`, `ApprovalGranted`, `StockSplitAccepted`)
- ☐ Set up shared Postgres schema + seed script early so both sub-teams can work in parallel against real data

PHASE 3 — Data Model (LLD) — Core Entities



User (id, name, email, password_hash, role, created_at)
Customer (id, name, tier[Bronze/Silver/Gold], email, portal_login)
Product (id, name, category, price, unit, tax_pct, description)
ProductVariant (id, product_id FK, attribute, value, extra_price)
PriceList (id, customer_tier, currency, price_rule)
DiscountTier (id, tier_name, max_discount_pct)
CategoryDiscountLimit (id, category, max_discount_pct)
ApprovalChainRule (id, discount_range_min, discount_range_max, requires_manager, requires_finance)
Quotation (id, customer_id FK, sales_rep_id FK, status, blended_risk_score, created_at, updated_at)
QuotationLine (id, quotation_id FK, product_id FK, qty, unit_price, discount_pct, line_limit_pct, status)
Approval (id, quotation_id FK, step[SalesManager/Finance], reviewer_id FK, action, note, timestamp)
Warehouse (id, name, location)
Stock (id, warehouse_id FK, product_id FK, qty_available, qty_reserved)
FulfillmentSplit(id, quotation_id FK, warehouse_id FK, qty_fulfilled, shipment_cost)
SubscriptionPlan(id, name, cycle[monthly/quarterly/yearly], proration_rule)
Subscription (id, quotation_id FK, plan_id FK, status, next_bill_date)
Invoice (id, quotation_id FK, type[one-time/recurring], amount, status, due_date)
Payment (id, invoice_id FK, amount, method, paid_at)
DealHealthFlag (id, quotation_id FK, flag_type[stalled/anomaly/slippage], detected_at, resolved)
UpsellRule (id, base_product_id FK, suggested_product_id FK, min_margin_pct, promo_active)
AuditLog (id, entity_type, entity_id, user_id FK, action, reason, timestamp)

Checklist:

- ☐ Draw this as an ERD (use `mermaid.js erDiagram` or a whiteboard tool) and get both sub-teams to sign off before coding
- ☐ Write seed data for: 1 warehouse pair, 1 discount tier set, 1 subscription plan, 3–4 products (mix of hardware/service/subscription) — this is what your 8-step test flow will run against

PHASE 4 — Sprint Plan (Design Thinking → Agile, Demo Each Time)

Each sprint should end in something **visibly demoable** to a mentor.

Sprint 1 (Foundation) — Team A leads, Team B shadows/reads schema

Auth (login/signup, role-based)

DB schema + seed data live

Product & Price List CRUD (Admin screens)

Warehouse + stock CRUD

Demo: log in as different roles, see role-based dashboards, view seeded products/stock

Sprint 2 (Quotation + Discount Governance) — Both teams active

Team A: Quotation Builder UI + API (add lines, apply discounts, save draft)

Team B: Discount Risk Engine — per-line limit check + blended score calculation, wired via event

(`QuotationUpdated`) → risk engine → (`ApprovalRequired`) event if breached)

Demo: build a quote, give an over-limit discount, watch it auto-flag for approval — no manual trigger

Sprint 3 (Approval + Fulfillment)

Team A: Approval screen (approve/reject/return), audit trail, Warehouse split screen

Team B: Auto-split suggestion logic (minimize shipment count given stock spread)

Demo: approve a flagged quote → see suggested warehouse split → accept or override

Sprint 4 (Billing + Portal)

Team A: Subscription/Billing screen (one-time + recurring lines), Customer Portal negotiation screen

Team B: Proration logic; portal negotiation re-triggering approval when terms breach threshold

Demo: confirm order with mixed billing → open portal as customer → request bigger discount → watch it re-enter approval automatically

Sprint 5 (Deal Health + Reporting + Polish)

Team A: Reporting screen with filters, PDF/XLS export

Team B: Deal Health Dashboard — stalled deal detection, discount anomaly detection (vs rep's own average), delivery slippage flags

Demo: full 8-step Quick Test Flow end to end, live

Checklist per sprint:

- ☐ Sprint planning: 15 min, pick stories from backlog
- ☐ Daily 10-min sync between the two sub-teams (event contracts must stay in sync!)
- ☐ End-of-sprint demo rehearsal before mentor visit
- ☐ Update the architecture/data-flow diagrams if anything changed

PHASE 5 — Integration, Testing & Demo Prep

- ☐ Run the full 8-step Quick Test Flow end-to-end, twice
- ☐ Prepare the "2 full flows" required for the 5-minute demo (suggest: Flow 1 = discount breach → approval → fulfillment split; Flow 2 = mixed billing → portal negotiation → re-approval → payment)
- ☐ Finalize the 1-page architecture diagram (use the HLD provided) for submission
- ☐ Write the "what we'd build next" note (multi-currency, ML-based anomaly detection, notification system, microservice extraction)

Frontend → API layer: REST calls (JSON), JWT in headers for auth

API layer → Core services: direct function calls within the same module boundary only

Core service → Core service: never direct — always via the event bus

Core service → Smart layer: event bus only (publish/subscribe)

Smart layer → Core service (feedback, e.g. "approval required"): also via event bus, core service subscribes and reacts (e.g. Approval service creates a record when it hears `ApprovalRequired`)

All services → Database: each module owns its own tables; cross-module reads go through that module's service layer, not raw joins, to keep modules swappable later

Frontend gets updates: either polling REST endpoints, or WebSocket push for live status (margin updates, approval status changes)

This is the same pattern used in the two diagrams shared above — refer back to those for the visual reference during your architecture walkthrough with mentors.